For bkhack, the user gets to familiarize with the concept of *stream-based programming*. Indeed, visitors of the site will often catch glance of command hints and tips as they are littered about the user interface. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Stream programming in bkhack is sh pipelining [1], with certain flavorful differences. The most important difference is that, instead of dealing with textual data and lines and files, here we deal with abstract post data and post counts. Consider

```
feed | split -c 10
```

where `split` has the option `-c NUM` where `NUM` is the count size of each chunk. Conventionally, the POSIX `split` command accepts the option `-l NUM` where `NUM` is line number size of chunk.

Another difference is that there is no filesystem, no concept of space. There is a command for every page, but there's no way to navigate "relatively" such as going "back" or "forth"; in other words, there's no `cd` command.

Shell and kernel

The original sh language is a user interface for the system shell, which itself is an interface for the kernel, as part of the shell-kernel architecture [2]. In comparison, the bkhack shell language is highly abstract and doesn't need a stand-in kernel analogy: at most, the bkhack website's command system emulates the shell-kernel architecture.

Pipeline

Introduced by Doughlas, pipelining enables commands to compose with each other parsibly simply via textual data. In this composition, there are relationships between commands to form the pipeline by parts.

The *source* is the start of the pipeline, it exclusively produces output. The *sink* is the end of the pipeline, it exclusively consumes the input. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere. The rest are *cantrips*–commands that don't produce output but instead apply an effect onto the current environment.

It's worth noting that, even on the conceptual level, the evaluation order of the part commands is that all commands start simultaneously when a pipeline runs.

For the bkhack shell language, pipelining is first-class citizen. The grammar expects all commands to unequivocally form a pipeline, as evident by Table 1, and multiple groups of commands will connect to form branching pipelines.

Function

A reusable pipeline or grouping of commands would be called a *function*. In the bkhack shell language, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

Grammar

The grammar of the bkhack shell language, given in EBNF form in Table 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.

<program>	::=	<command> (`|` <program>)$_?$	*Simple program*
		`{` <program> <end of seq> `}` (`|` <program>)$_?$	*Composite program*
		`(` <program> `)` (`|` <program>)$_?$	*Subshell*
<command>	::=	<word> (<command>)$_?$	
<end of seq>	::=	`;`	
<word>	::=	*Omitted*	
<seq connective>	::=	`;`	*Default continuation*
		`&&`	*Success continuation*

Table 1: Grammar for the bkhack shell language

$$\frac{}{\text{nil } () : \text{Program } T}\,\text{Zero} \qquad \frac{x : \text{Cmd } T \quad x' : \text{Program } T}{||(x, x') : \text{Program } T}\,\text{Cons}$$

$$\frac{t : \text{Program } A}{\$\%(t) : \text{Cmd } A}\,\text{Substitute} \qquad \frac{\text{t} : \text{Program A}}{\text{obs}(t) : \text{A}}\,\text{Observe}$$

Figure 1: Typing for the bkhack shell language

Notice how the command rule do not distinguish parts into flags or values which are expected from most real-world usage. Indeed, the responsibility of interpreting these parts, a.k.a. *options*, is left to the higher-level *option parser*, such as POSIX `getopts`.

Typing

The typing of the bkhack shell language, given in Figure 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri. This is useful when shell commands have to be embedded in a statically-typed programming language.

Reference

[1] “Shell Command Language.” IEEE, The Open Group. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html

[2] “???”